

Week 1 R, Getting Started with R Studio, dataframes, and plotting

2025-07-25

Disclaimer

This series of exercises is designed to focus on helping you take your first steps in data visualisation with R using a practical approach. We will only cover basic programming techniques, so please use in conjunction with other resources, and R documentation. LLMs such as ChatGPT while very useful are not recommended, as you cannot know if the output is sensible, if you have not learnt the basics of how to use R.

Why Choose R?

Your choice of tools should depend on:

- Industry standards in your target field, along with colleague expertise
- Your experience ie the speed of delivering solutions vs learning new tools
- Strengths and weaknesses of the toolset

Strengths

- Built-in statistical capabilities: R was designed for statistics, with base functions for descriptive stats, regression, and data frames (originating from S). This means it also has efficient syntax for statistical analysis, often requiring less code than other languages.
- C-Like (almost): R has some similarities with C family languages, making it more natural to move to and from these languages.
- Domain Specific language, R's focus on statistical programming makes it highly effective for analysis and visualization. However, it works best when combined with a broader toolbox of technologies, freeing you up to choose the best tool for the job, and encouraging a more versatile skill set.
- RStudio IDE: Excellent for data exploration, plotting, variable tracking along with fantastic support for integration with Markdown, Python, JavaScript, D3, along with more.
- Standard: R has been the standard for statistical programming, this means it has extensive base functionality for statistic, along with a good selection of packages of which can be trusted to be correct.
- Publication-Quality Visualisations and Documentation: R's libraries such as markdown and ggplot2 are designed to produce high-quality customization visualisations and documentation, ensuring it is ideal for quickly creating academic or professional publications.

Weaknesses

- AI/ML support is limited: While R offers market leading packages like Keras (a Python wrapper) and Torch for R (calling Pytorch's C++ directly), they often lag behind their Python counterparts and can sometimes be challenging to install and maintain. Older R-specific libraries (e.g., Rneuralnet) are useful for learning but unsuitable for modern deep learning. Most cutting-edge AI solutions are developed in Python, though R can integrate with Python for combined workflows.
- Networking and Web apps: R Shiny is great, but does require a Shiny Sever backed, this is not only less common than solutions such as PHP and Node, but potentially trickier.
- Performance: A complex topic, but generally comparable to Python, although it might lag behind in certain tasks. Certainly not a performance targeted language such as Julia or C++.
- Domain Specific Language: Working in R alone will add some limitations to what you are able to produce, you will almost certainly need to learn other technologies and tools.

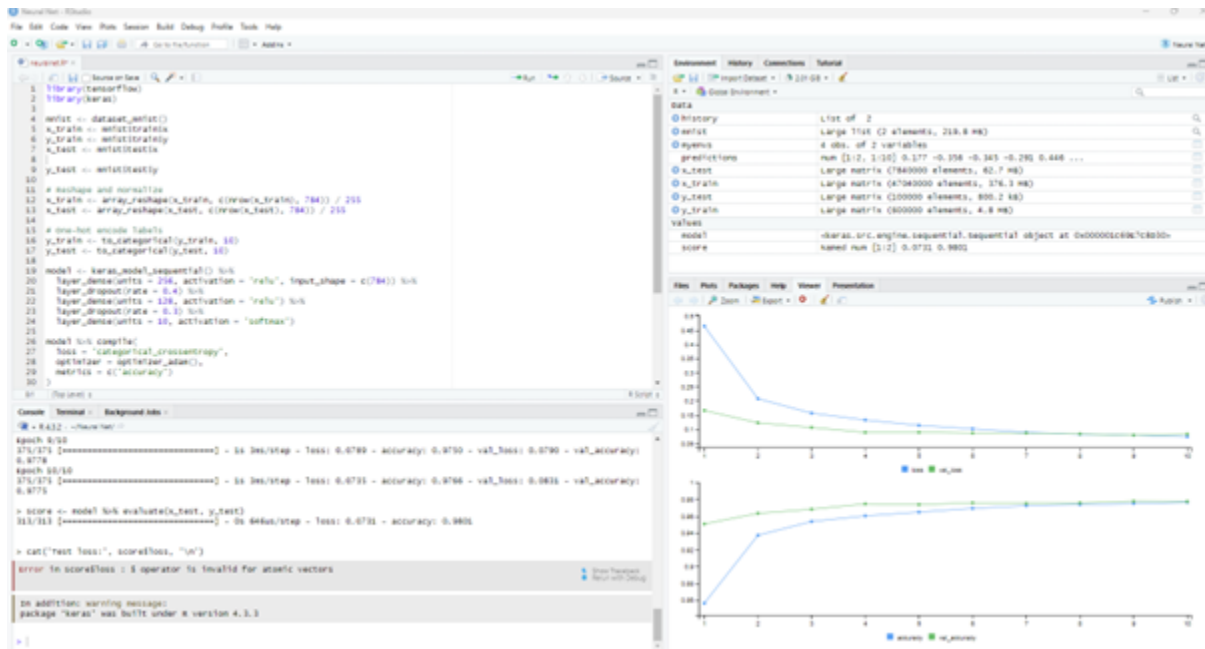
Contents

There is no requirement to complete these labs for this module. They only aim to give introductions to tools that you might wish to use.

- Lab 0: Introduction and getting started.
- Lab 1: Preprocessing and creating a bar chart with plot
- Lab 2: Scatter plots with ggplot2
- Lab 3: Preprocessing with DPLYR and Visualisation with ggplotly
- Lab 4: Conditional logic and subplots with plotly
- Lab 5: Heatmaps and absolute positioning.
- Lab 6: Annotations
- Lab 7 (Intermediate): Maps
- Lab 8 (Advanced): Shiny

Getting Started

Navigate to R Studio. Follow instructions to install the latest suitable version of R (at time of writing 4.5.1) for your environment. Then return to the link and install the latest R Studio.



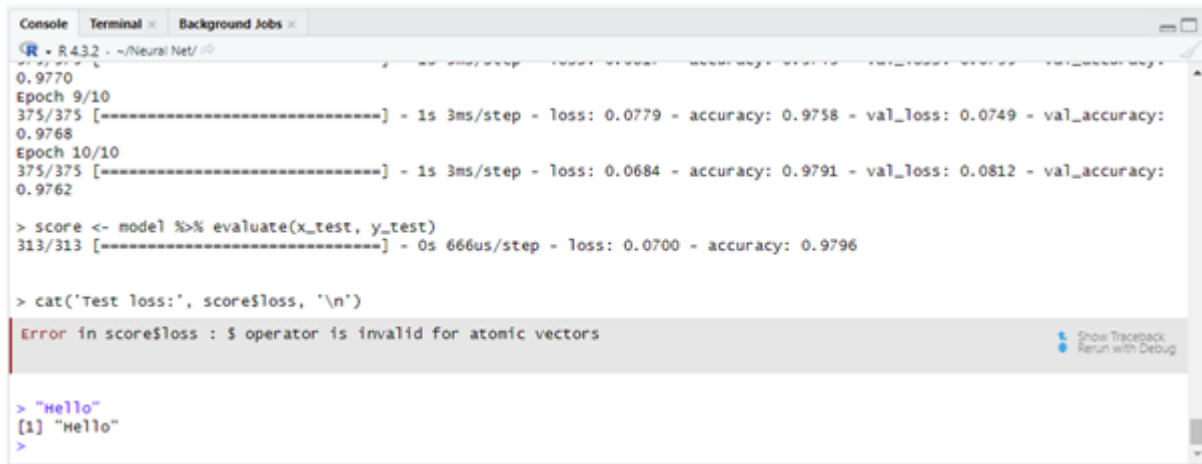
This image shows a standard R view.



The script window if not already open can be opened by selecting File -> New File -> R Script. You can use the 'Run' button to run a line of code, or run the whole script with 'Source' (windows default hot keys are ctrl+shift+enter).

The console windows allows you to directly enter code. Most of the GUI operations can be also completed here. To install packages for instance you can type `install.packages("your package name")`

The environment window is an extremely useful tool, Here you can investigate stored data, and import data sets along with other features.



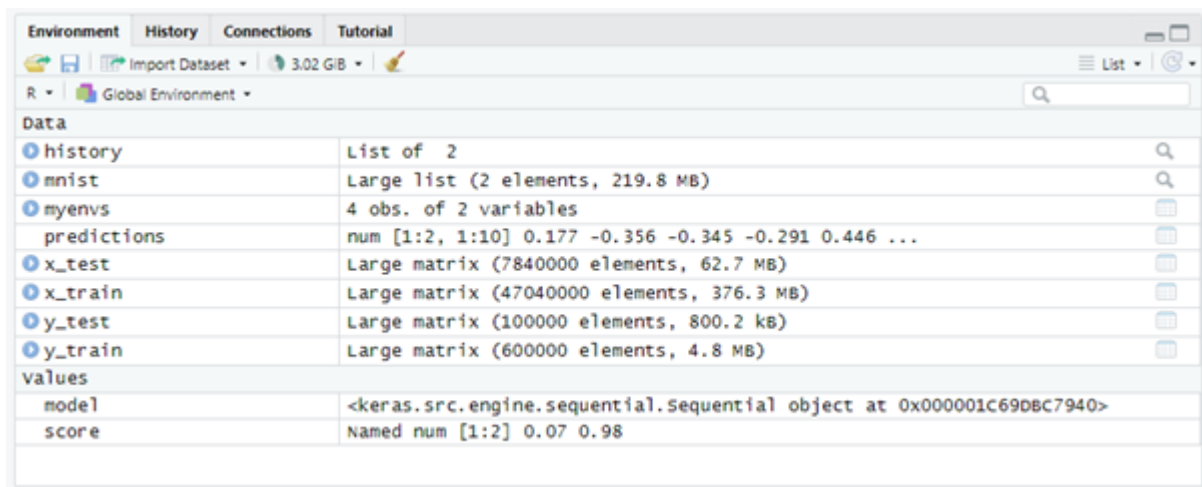
```
R - R 4.3.2 - ~/Neural Net/
0.9770
Epoch 9/10
375/375 [=====] - 1s 3ms/step - loss: 0.0779 - accuracy: 0.9758 - val_loss: 0.0749 - val_accuracy:
0.9768
Epoch 10/10
375/375 [=====] - 1s 3ms/step - loss: 0.0684 - accuracy: 0.9791 - val_loss: 0.0812 - val_accuracy:
0.9762

> score <- model %>% evaluate(x_test, y_test)
313/313 [=====] - 0s 666us/step - loss: 0.0700 - accuracy: 0.9796

> cat('Test loss:', score$loss, '\n')
Error in score$loss : $ operator is invalid for atomic vectors
Show Traceback
Rerun with Debug

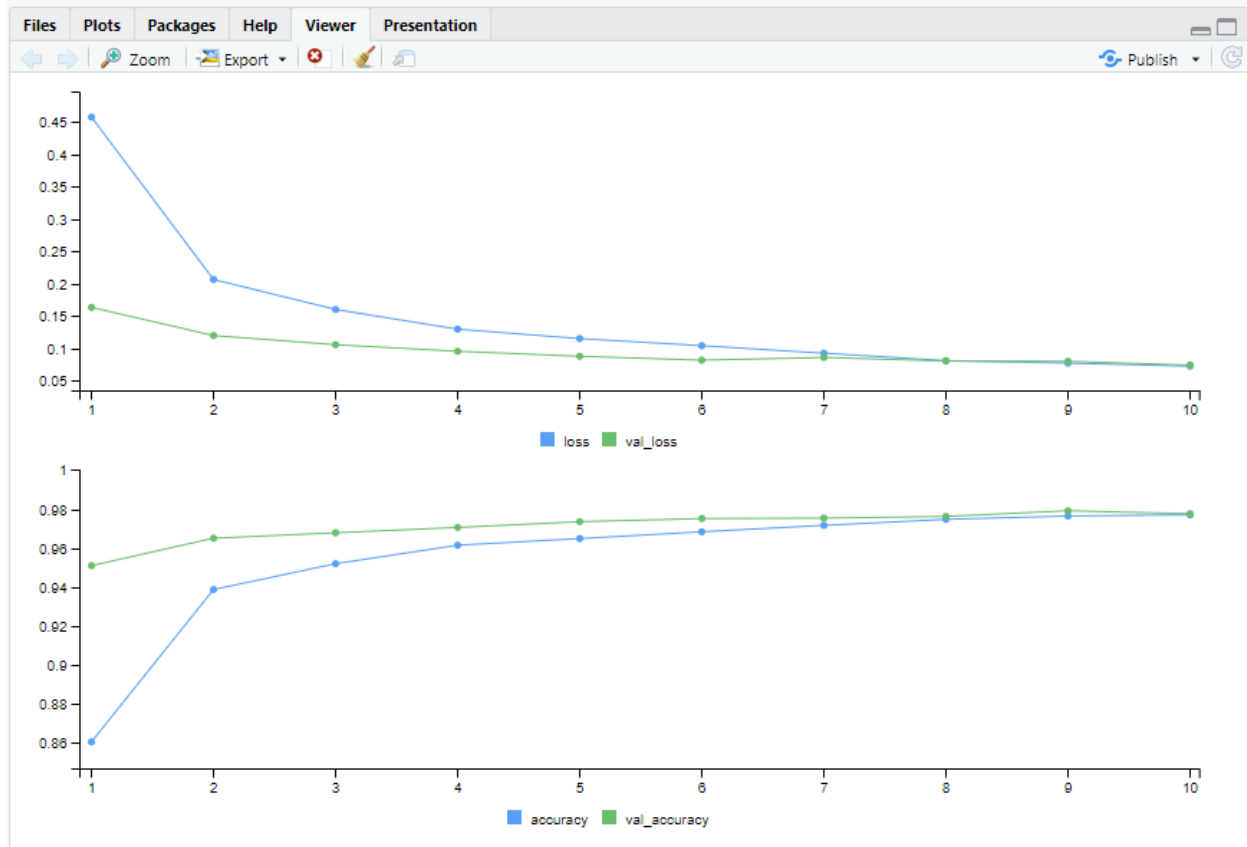
> "Hello"
[1] "Hello"
>
```

Figure 1: Console



Environment	
R - Global Environment	
Data	
history	List of 2
mnist	Large list (2 elements, 219.8 MB)
myenvs	4 obs. of 2 variables
predictions	num [1:2, 1:10] 0.177 -0.356 -0.345 -0.291 0.446 ...
x_test	Large matrix (7840000 elements, 62.7 MB)
x_train	Large matrix (47040000 elements, 376.3 MB)
y_test	Large matrix (100000 elements, 800.2 kB)
y_train	Large matrix (600000 elements, 4.8 MB)
values	
model	<keras.src.engine.sequential.Sequential object at 0x000001c69d8c7940>
score	Named num [1:2] 0.07 0.98

Figure 2: Intro to Enviroment

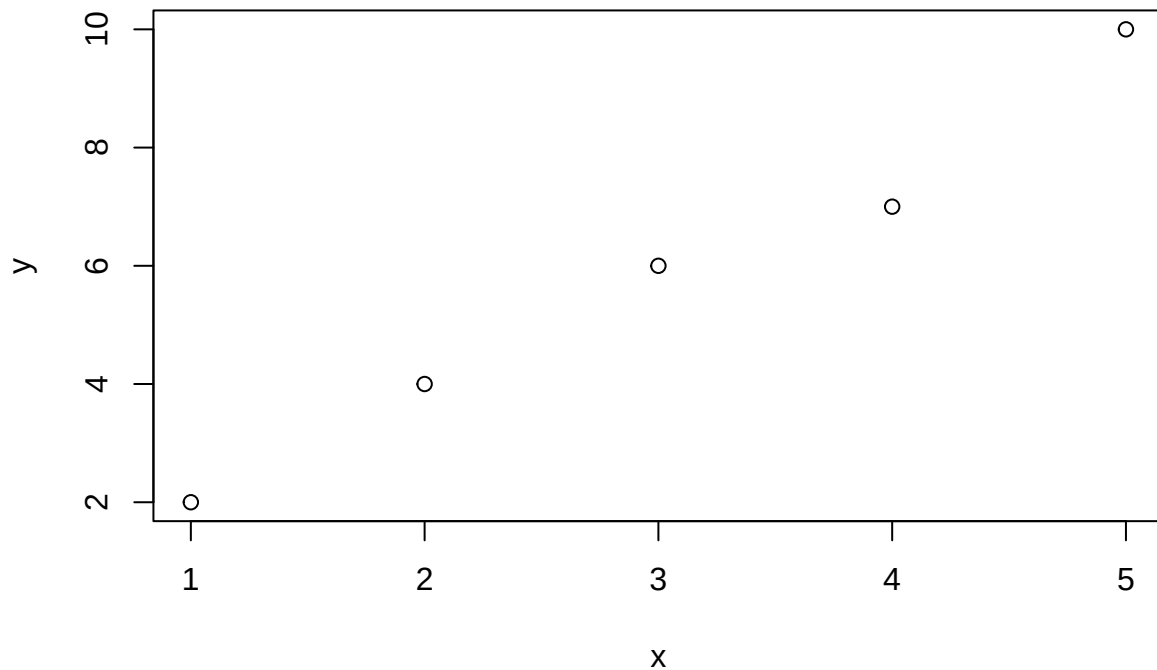


The plot window allows you to view some plots and visualisations that you create. This can be extremely handy for quick data exploration. You can also use this window for other tasks such as managing packages and viewing help documentation.

Vectors and Dataframes

Open a new script file (or run in console)

```
x <- c(1,2,3,4,5)
y <- c(2,4,6,7,10)
plot(x,y)
```



This code combines the numbers in the brackets to create vectors x and y.

Environment

History

Connections

Tutorial

Import Dataset

165 MiB

List

R

Global Environment

Data

runs_only

1090 obs. of 57 variables

values

x

num [1:5] 1 2 3 4 5

y

num [1:5] 2 4 6 7 10

Figure 3: vectors

The can be explored in the environment window. Allowing you to keep track of assigned variables.

plot is r's generic plotting library, we will largely not be using it for our final visualisations, but it is quick and easy to use for data exploration.

```
df <- data.frame(x, y)
print(df)
```

```
##   x y
## 1 1 2
## 2 2 4
## 3 3 6
```

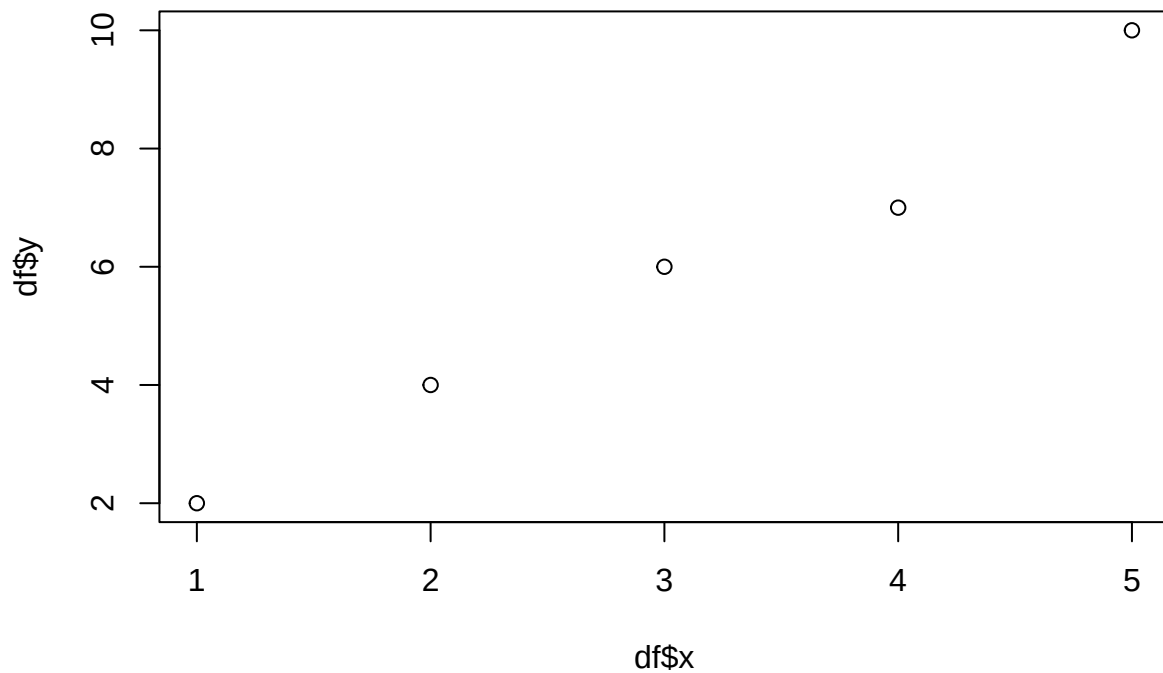
```
## 4 4 7
## 5 5 10
```

Dataframes are an extremely useful part of r, managing two dimensional tabular data. You can print the data as above or select from the environment pane for a spreadsheet like view.

	x	y
1	1	2
2	2	4
3	3	6
4	4	7
5	5	10

You can plot the dataframe similarly

```
plot(df$x, df$y)
```



The dollar sign allows you to access the dataframe elements by name;

You can also use this method to do operations to the dataframe; much simpler to write than having to loop through the data.

```
df$y <- df$y+1
df$z <- df$x+df$y
print(df)
```

```
##   x  y  z
## 1 1  3  4
## 2 2  5  7
## 3 3  7 10
## 4 4  8 12
## 5 5 11 16
```

Along with filtering;

```
filtered <- df[df$y >5,]
print(filtered)
```

```
##   x  y  z
## 3 3  7 10
## 4 4  8 12
## 5 5 11 16
```

ie; filtered is a new dataframe that only contains rows where y is greater than 5, it takes the format [rows,columns].

A list of operators can be found on w3schools;

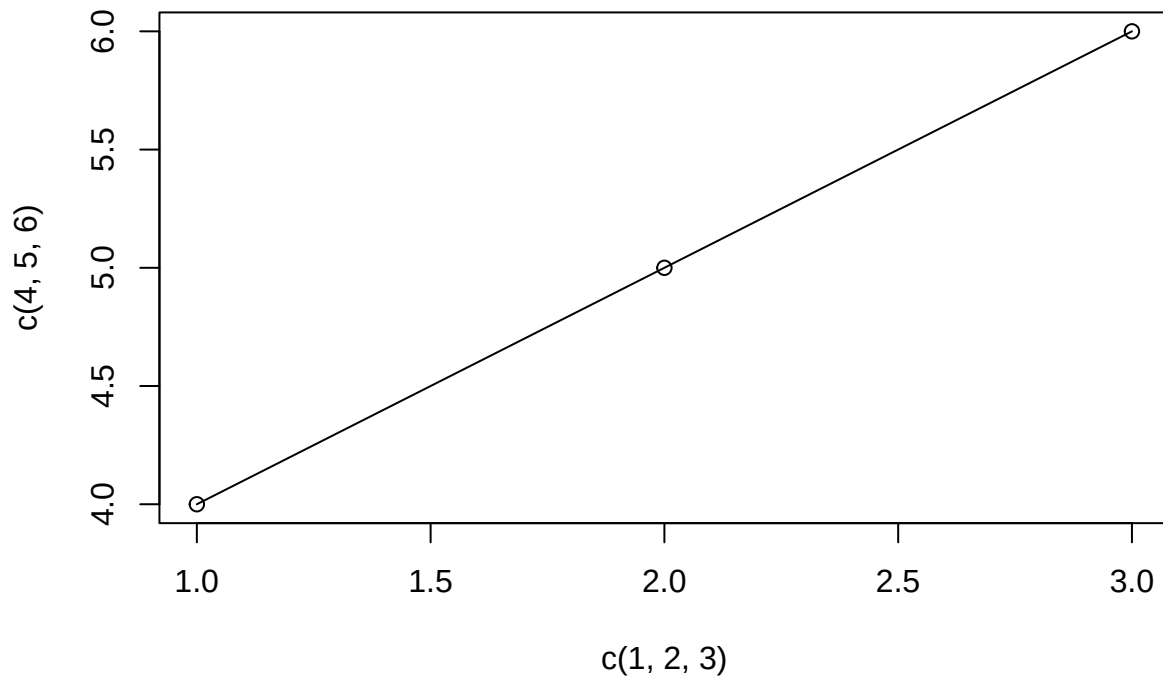
https://www.w3schools.com/r/r_operators.asp

Excercises

Note: You will need to research the named functions. Avoid chatGPT, use web resources and RDocumentation.

- 1) Create a data frame with two columns x and y where x has 10 values on the range -10 to 10, and $y = \cos(x)$. Plot the output.
- 2) Use lines() to add a line to the scatter chart that goes through the points.

```
# Example of adding a line to a plot
plot(c(1,2,3), c(4,5,6))
lines(c(1,2,3), c(4,5,6))
```

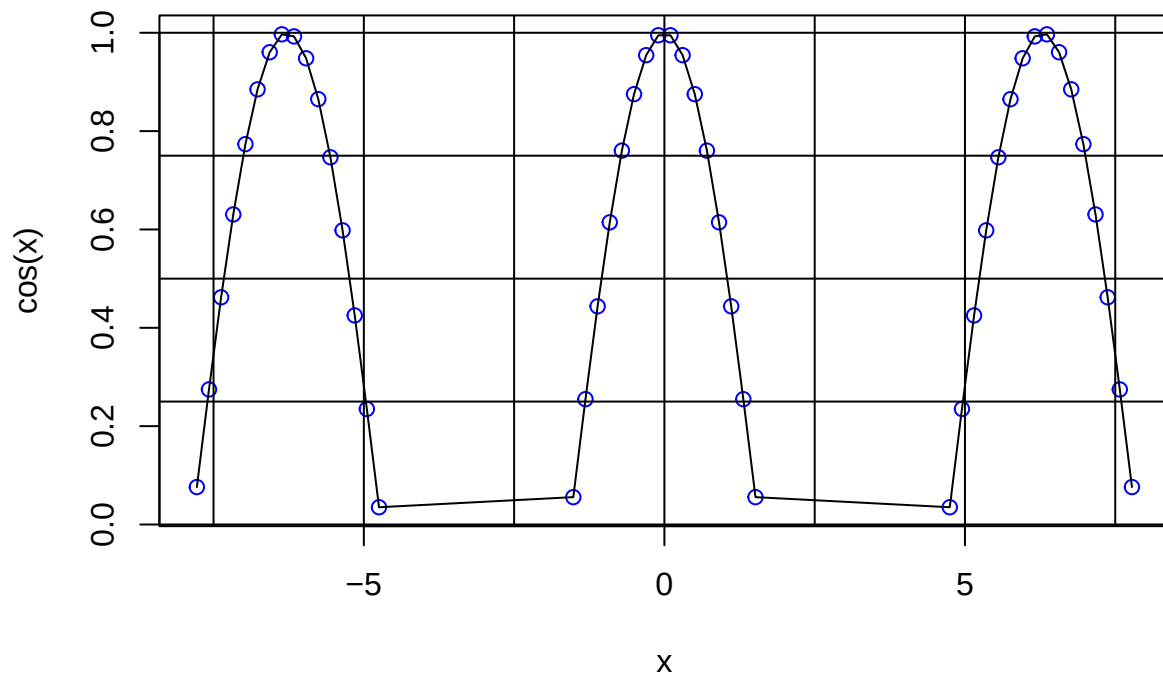
3) Use the `seq()` function to increase the number of x values to 100 still on the range -10 to 10.

```
#Example of using sequence
sequence <- seq(-1, 1, length.out = 10)
print(sequence)
```

```
## [1] -1.0000000 -0.7777778 -0.5555556 -0.3333333 -0.1111111  0.1111111
## [7]  0.3333333  0.5555556  0.7777778  1.0000000
```

- 4) Plot only positive values
- 5) Label axis and title chart
- 6) Make the scatter points blue
- 7) Bonus - find a method to grid lines at 2.5 intervals on the x axis.

Cosine Curve (Positive y Values)



Importing data

Importing data with R Studio is pretty straightforward. One method is to use the ‘Import Dataset’ button in the environment window.

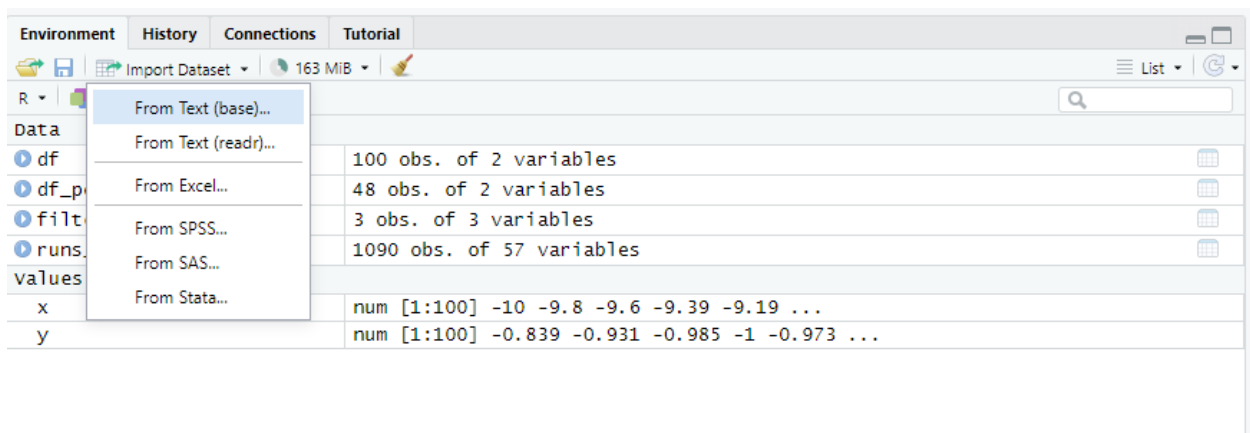
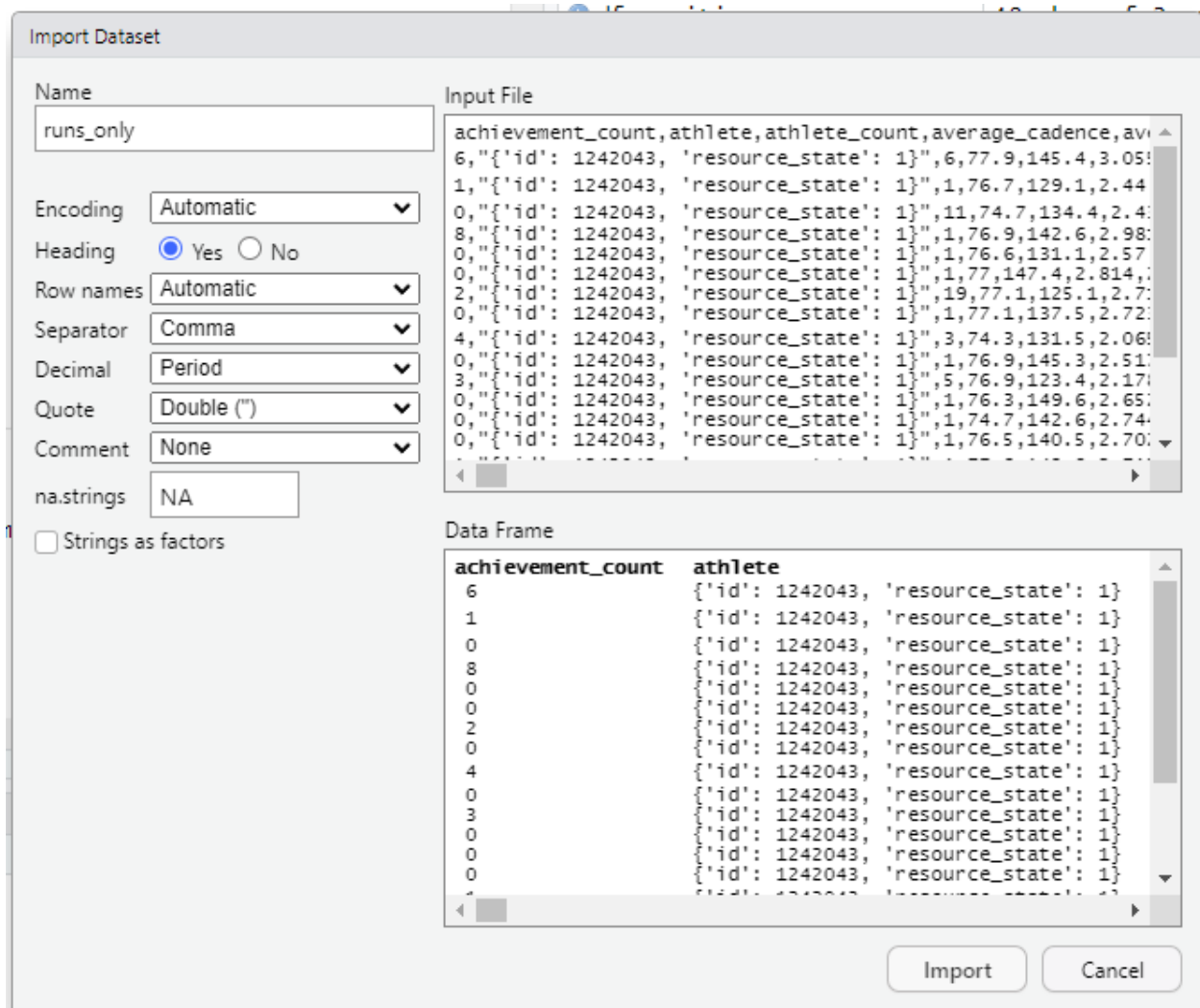


Figure 4: import

This will provide a few options you can explore. From Text (base) provides the most basic option requiring no additional dependencies. Using Martins runs_only.csv dataset go ahead and try it out;



You'll see options for reading the file. The file is a csv type file, basically plain text (you can open in any text editor) with each column separated by commas. Before importing you can check the windows to see if the data is being read correctly, in this case the file is very standard so you can directly then import.

You'll now see in your environment window as a dataframe;

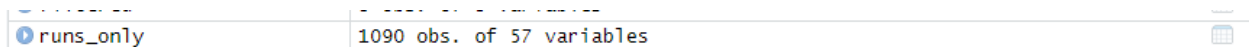


Figure 5: runsonly

app.R runs_only Week 1 Exercises R.Rmd* solution 1.R* df									
Filter Cols: << 1 - 50 >>									
	achievement_count	athlete	athlete_count	average_cadence	average_heart_rate	average_speed	average_temp	average_watts	comm
1	6	{'id': 1242043, 'resource_state': 1}	6	77.9	145.4	3.055	25	354.4	
2	1	{'id': 1242043, 'resource_state': 1}	1	76.7	129.1	2.440	30	292.6	
3	0	{'id': 1242043, 'resource_state': 1}	11	74.7	134.4	2.437	26	276.3	
4	8	{'id': 1242043, 'resource_state': 1}	1	76.9	142.6	2.981	24	341.9	
5	0	{'id': 1242043, 'resource_state': 1}	1	76.6	131.1	2.570	29	275.5	
6	0	{'id': 1242043, 'resource_state': 1}	1	77.0	147.4	2.814	26	329.8	
7	2	{'id': 1242043, 'resource_state': 1}	19	77.1	125.1	2.714	26	301.2	
8	0	{'id': 1242043, 'resource_state': 1}	1	77.1	137.5	2.723	28	313.5	
9	4	{'id': 1242043, 'resource_state': 1}	3	74.3	131.5	2.065	25	280.0	
10	0	{'id': 1242043, 'resource_state': 1}	1	76.9	145.3	2.517	30	288.3	
11	3	{'id': 1242043, 'resource_state': 1}	5	76.9	123.4	2.178	26	253.5	
12	0	{'id': 1242043, 'resource_state': 1}	1	76.3	149.6	2.652	28	305.8	
13	0	{'id': 1242043, 'resource_state': 1}	1	74.7	142.6	2.744	25	299.0	
14	0	{'id': 1242043, 'resource_state': 1}	1	76.5	140.5	2.702	27	315.7	
15	1	{'id': 1242043, 'resource_state': 1}	1	77.8	143.9	2.717	27	310.7	
16	4	{'id': 1242043, 'resource_state': 1}	1	78.0	138.8	2.774	25	316.7	
17	13	{'id': 1242043, 'resource_state': 1}	115	77.6	174.1	2.930	29	337.2	
18	1	{'id': 1242043, 'resource_state': 1}	6	76.8	141.3	2.476	21	309.4	

Figure 6: runsonlydf